



BANGLADESH UNIVERSITY OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING

EEE 312 — DIGITAL SIGNAL PROCESSING I LABORATORY

LEVEL-3, TERM-I · JANUARY 2026

Project Report

DIGITAL SIGNAL PROCESSING AND DEEP LEARNING-BASED URBAN CHANGE DETECTION USING SATELLITE IMAGERY

Submitted To:

Tanushri Medha Kundu

Lecturer, Department of Electrical and Electronic Engineering, BUET

Abrar Assaeem Fuad

Lecturer, Department of Electrical and Electronic Engineering, BUET

Submitted by

Group 04 — Section B2

Name	Student ID
Mustasin Rahman	2206104
Fahim Shahriyar	2206114
Md. Ahasan Habib Kawsar	2206119
Md. Kawsar Ahmed	2206122

Date of Submission: 2/10/2026

EEE 312 (January 2026)
Digital Signal Processing I Laboratory

Final Project Report

Section: B2 Group: 04

Digital Signal Processing and Deep Learning-Based Urban Change Detection Using Satellite Imagery

Course Instructors:

Tanushri Medha Kundu

Lecturer, Department of Electrical and Electronic Engineering, BUET

Abrar Assaeem Fuad

Lecturer, Department of Electrical and Electronic Engineering, BUET

Signature of Instructor: _____

Academic Honesty Statement:

IMPORTANT! Please carefully read and sign the Academic Honesty Statement, below. Type the student ID and name, and put your signature. *You will not receive credit for this project experiment unless this statement is signed in the presence of your lab instructor.*

"In signing this statement, we hereby certify that the work on this project is our own and that we have not copied the work of any other students (past or present), and cited all relevant sources while completed this project. We understand that if we fail to honor this agreement, we will each receive a score of ZERO for this project and be subject to failure of this course."

Signature: _____

Full Name: Mustasin Rahman

Student ID: 2206104

Signature: _____

Full Name: Fahim Shahriyar

Student ID: 2206114

Signature: _____

Full Name: Md. Ahasan Habib Kawsar

Student ID: 2206119

Signature: _____

Full Name: Md. Kawsar Ahmed

Student ID: 2206122

Table of Contents

1	Abstract	1
2	Introduction	1
3	Design.....	3
3.1	Problem Formulation (PO(b)).....	3
3.1.1	Identification of Scope	3
3.1.2	Literature Review	3
3.1.3	Formulation of Problem	4
3.1.4	Analysis	4
3.2	Design Method (PO(a))	5
3.2.1	Data Preparation and Preprocessing.....	5
3.2.2	DSP Front-End	6
3.2.3	Siamese U-Net Architecture.....	7
3.2.4	Spatial-Temporal Attention and Siam-Concat Fusion	8
3.2.5	Loss Function	8
3.2.6	Training Procedure	9
3.2.7	Inference, Thresholding and Post-Processing	9
3.2.8	Evaluation Metrics	10
3.3	Source Code (Core Modules)	10
4	Implementation.....	11
4.1	Description.....	11
5	Design Analysis and Evaluation	14
5.1	Novelty	14
5.2	Design Considerations (PO(c)).....	15
5.2.1	Considerations to public health and safety.....	15
5.2.2	Considerations to environment.....	15
5.2.3	Considerations to cultural and societal needs.....	15
5.3	Investigations (PO(d))	15
5.3.1	Design of Experiment.....	15
5.3.2	Data Collection	16
5.3.3	Results and Analysis	16
5.3.4	Interpretation and Conclusions on Data	22
5.4	Limitations of Tools (PO(e))	23
5.5	Ethical Issues (PO(h)).....	23
6	Reflection on Individual and Team work (PO(i)).....	24

6.1	Individual Contribution of Each Member.....	24
6.2	Mode of TeamWork	24
6.3	Diversity Statement of Team	24
6.4	Log Book of Project Implementation	25
7	Communication to External Stakeholders (PO(j)).....	25
7.1	Executive Summary	25
7.2	User Manual.....	26
7.3	Github Link.....	27
7.4	YouTube Link.....	27
7.5	Google Drive Link (Alternative Video Link).....	27
8	Future Work (PO(l))	27
9	References	28

1 Abstract

Rapid urban growth creates a need for timely mapping of building construction and demolition from bi-temporal high-resolution imagery. This project develops and experimentally evaluates a DSP-informed deep-learning pipeline for pixel-level building-change detection under constrained computing resources. Each RGB image is converted to luminance, smoothed with a 5×5 Gaussian filter ($\sigma=1$), and processed by Canny edge detection with thresholds 50/150. The resulting binary edge map is standardised and concatenated with RGB as a fourth input channel. A shared ResNet-34 encoder with spatial-temporal attention, Siam-Concat fusion, and an edge-guided U-Net decoder produces the change map. Training uses native-resolution 256×256 random crops with change-focused sampling and synchronized geometric augmentation; full 1024×1024 scenes are reconstructed from overlapping 256×256 tiles using cosine-weighted blending and test-time augmentation.

On the 128 unseen LEVIR-CD test pairs, the RGB + Canny reference configuration achieved 91.17% F1 and 83.78% IoU, while test-time augmentation increased F1 by 0.40 points. Relative to the first prototype, the corrected pipeline improved F1 by 14.05 points; because several engineering corrections were introduced together, this gain is treated as a combined pipeline improvement rather than attributed to a single component. A retrained RGB-only ablation achieved 91.63% F1, showing that the fixed Canny channel did not provide a measurable performance benefit under this setup. Because only one training seed was used, this small difference is not treated as conclusive evidence against explicit edge features. On WHU-CD, the 4-channel configuration achieved 82.85% F1. Purbachal, Dhaka image pairs were evaluated qualitatively because ground-truth labels are unavailable. The complete system is deployed as a desktop application with interactive thresholding and polygon post-processing.

Key results at a glance

91.2 % F1-score (LEVIR- CD test)	83.78 % IoU (LEVIR- -CD test)	91.8 % Precision (LEVIR- CD test)	90.6 % Recall (LEVIR- CD test)	82.85 % F1 on unseen city (WHU- CD)	+14.05 pts F1 gain over First prototype (After Debugging)	+0.40 pts F1 from TTA	+0.46 pts RGB- only vs RGB+ Canny
--	--	---	--	---	---	------------------------------------	--

2 Introduction

Bangladesh is urbanising rapidly. Dhaka and its surrounding municipalities add new residential blocks, factories and roads every year, often faster than planning authorities can survey them. Knowing *where* and *when* buildings appear is essential for detecting unauthorised construction, monitoring encroachment on wetlands and flood-flow zones, updating utility and census maps, and assessing damage after floods or cyclones. Very-high-resolution satellite imagery is now available regularly, so the bottleneck is no longer data acquisition but its analysis [1].

Change detection compares two images of the same place taken at times T_1 (before) and T_2 (after) and produces a binary change map. Although the task sounds simple, it is a complex engineering problem:

- ❖ **Depth of knowledge (WP1):** it requires 2-D signal processing (filtering, convolution, gradient operators), statistics (class imbalance, thresholds) and deep learning (convolutional networks, attention, optimisation).
- ❖ **Conflicting requirements (WP2):** precision versus recall (false alarms versus missed buildings), resolution versus memory, and accuracy versus inference time on a laptop GPU.
- ❖ **Depth of analysis (WP3):** there is no closed-form solution; the design must be found through modelling, experiments and ablations.
- ❖ **Interdependence (WP7):** preprocessing, network architecture, loss function, threshold and post-processing interact; a defect in one stage (for example input scaling) silently degrades the whole system.

Real imagery adds further difficulty: illumination, season, shadows and sensor colour change between dates even when nothing is built (**Pseudo-changes**); only about 4.6 % of pixels actually change (**extreme class imbalance**); and at 0.5 m/pixel a house is only 10–20 pixels wide (**small objects**). Figure 1 shows a typical pair from the LEVIR-CD dataset.

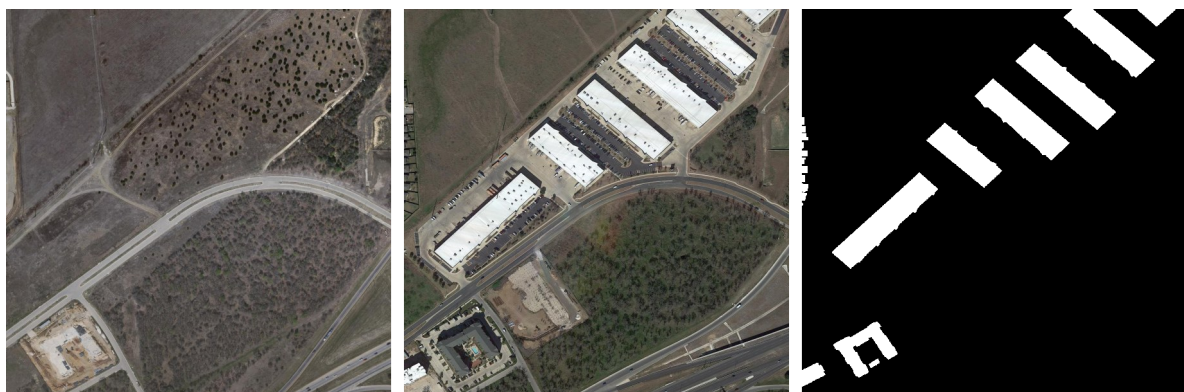


Figure 1: A LEVIR-CD image pair (test_102): (left) T_1 before, (middle) T_2 after, (right) ground-truth change mask (white = new buildings).

Possible alternative solutions include: (i) manual photo-interpretation, which is accurate but slow and expensive; (ii) pixel-based methods such as image differencing, image ratioing and change-vector analysis, which are fast but very sensitive to illumination ^[1]; (iii) PCA/k-means and object-based methods, which need hand-tuned features; (iv) Siamese convolutional networks ^[2]; and (v) attention and transformer networks ^{[3], [4], [5]}, which are accurate but computationally heavy. We chose a hybrid of (ii)–(v): an explicit DSP edge front-end feeding a lightweight Siamese CNN with attention that can be trained on a laptop, and we tested experimentally whether the DSP stage helps.

3 Design

3.1 Problem Formulation (PO(b))

3.1.1 Identification of Scope

- **Input:** two co-registered RGB satellite images of the same area (T_1 , T_2), any size; primary resolution 0.5 m/pixel.
- **Output:** a per-pixel change-probability map, a binary change mask, building-footprint polygons and the changed-area percentage.
- **Change type:** building construction and demolition (the classes annotated in LEVIR-CD); roads and vegetation are out of scope.
- **Data:** LEVIR-CD for training, validation and testing [3]; WHU-CD [6] (Christchurch, 0.3 m/pixel, labelled) for a cross-dataset test; real Bangladeshi imagery (Purbachal, Dhaka) for a qualitative domain-shift test.
- **Platform constraint:** training and inference on an Apple-silicon laptop GPU (pytorch MPS backend), no cloud GPU.
- **Deliverables:** trained model, evaluation reports, a desktop GUI, public source code and this report.

3.1.2 Literature Review

Early change detection relied on algebraic operations on co-registered images, image differencing, ratioing and change-vector analysis, followed by a threshold [1]. These methods are fast but respond to any radiometric difference, so seasonal and illumination changes produce many false alarms. Edge-based DSP operators, the Marr–Hildreth Laplacian-of-Gaussian [7] and the Canny detector [8], capture structural boundaries that are more stable than raw intensities.

Deep learning changed the field. Daudt *et al.* introduced fully convolutional Siamese networks (FC-EF, FC-Siam-conc, FC-Siam-diff) in which weight-sharing encoders process both dates and the decoder operates on concatenated or differenced features [2]. Chen and Shi proposed STANet, with a spatial-temporal attention module (BAM), and released the LEVIR-CD benchmark [3]. Transformer-based models such as BIT [4] and ChangeFormer [5] model global context and currently reach about 89–90 % F1 on LEVIR-CD, at a higher computational cost. Edge supervision with class-balanced losses, as in holistically-nested edge detection (HED) [9], is known to sharpen predicted boundaries. Table 1 summarises published results on the LEVIR-CD test set.

Table 1: Published LEVIR-CD test results (values as reported in [5]) and this work. Our figure uses sliding-window inference with test-time augmentation on the same 128 test images, so the comparison is indicative.

Method	Type	F1 (%)	IoU (%)
FC-EF [2]	Early-fusion CNN	83.40	71.53
FC-Siam-diff [2]	Siamese CNN	86.31	75.92
STANet [3]	Siamese CNN + attention	87.26	77.40
BIT [4]	CNN + transformer	89.31	80.68
ChangeFormer [5]	Transformer	90.40	82.48
This work (final model)	Siamese CNN + attention + DSP edges	91.17	83.78

Gap addressed: most published models do not use explicit DSP features, and most assume server-class GPUs. We investigate whether a classical, interpretable edge channel can be fused with a pre-trained CNN, and design the whole pipeline to train and run on a laptop.

3.1.3 Formulation of Problem

Let $I_1, I_2 \in \mathbb{R}^{H \times W \times 3}$ be the co-registered images and $Y \in \{0,1\}^{H \times W}$ the ground-truth change mask. We seek a function f_θ with learnable parameters θ that outputs a change probability for every pixel:

$$P = f_\theta(\phi(I_1), \phi(I_2)) \in [0,1]^{H \times W}, \quad \hat{Y} = 1[P \geq t] \quad (1)$$

where $\phi(\cdot)$ is the DSP front-end that maps an RGB image to a four-channel tensor (RGB + edge) and t is a decision threshold. θ is found by minimising a loss $L(P, Y)$ on the training set, and t is selected on the validation set to maximise the F1-score. The design objectives are:

- (1) maximise F1 and IoU on unseen test images;
- (2) keep false alarms and misses balanced;
- (3) train within a few hours and predict a 1024×1024 scene within seconds;
- (4) provide an interactive tool for non-experts.

3.1.4 Analysis

- ❖ **Class imbalance:** in a sample of 60 LEVIR-CD training images only 4.6 % of pixels are changed, and 18 % of images contain no change at all. A plain pixel-wise loss would be dominated by the background, so an overlap-based (Dice) loss and change-focused sampling are required.
- ❖ **Resolution:** the dataset images are 1024×1024 pixels. Down-sampling them to 256×256 (our first prototype) shrinks a 16-pixel house to 4 pixels. We therefore train on native-resolution crops and predict full images with a sliding window.
- ❖ **Attention cost:** full self-attention over two 64×64 feature maps would need an (8192×8192) attention matrix per image pair. Pooling the keys and values by 4 reduces it to 8192×512 , which fits in laptop memory.
- ❖ **Input scaling:** RGB pixel values lie in $[0, 255]$ while a binary edge map lies in $\{0, 1\}$. Without standardisation the edge channel is about 255 times smaller than the colour channels and is effectively ignored by the network.
- ❖ **Edge statistics:** measured on 120 LEVIR-CD images, Canny marks 12.1 % of pixels

as edges, so the edge channel has mean 0.12 and standard deviation $\sqrt{(0.121 \times 0.879)} \approx 0.33$; these values are used to standardise it.

3.2 Design Method (PO(a))

Figure 2 shows the complete design flow. Each stage applies specific knowledge of mathematics, signal processing and machine learning, described in the following subsections.

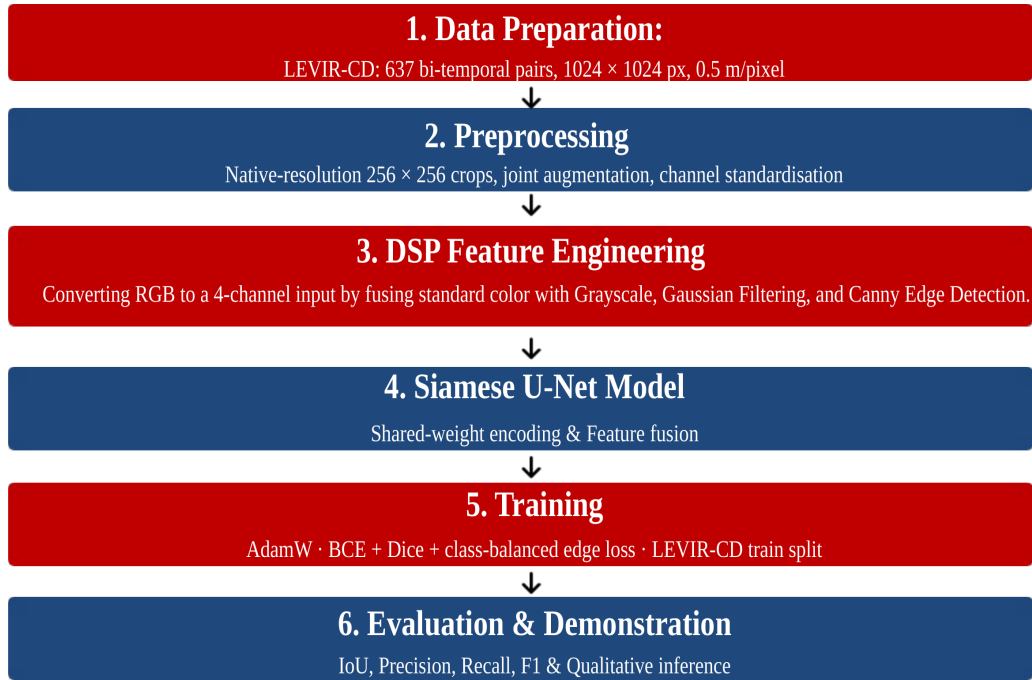


Figure 2: Overall design flow of the urban change-detection system.

3.2.1 Data Preparation and Preprocessing

LEVIR-CD contains 637 bi-temporal Google Earth image pairs of 1024×1024 pixels at 0.5 m/pixel, collected over several cities in Texas between 2002 and 2018, with pixel-level building-change annotations ^{[3], [10]}. We use the official split of 445 training, 64 validation and 128 test pairs (Table 2).

Table 2: LEVIR-CD dataset split.

Split	Image pairs	Pixels per image	Use
Train	445	1024×1024	Weight learning (random 256×256 crops)
Validation	64	1024×1024	Model selection and threshold choice
Test	128	1024×1024	Final evaluation only (never used for tuning)

During training, four random 256×256 crops are drawn from every image in each epoch; 30 % of the crops are centred on a randomly chosen changed pixel to counter class imbalance. Horizontal and vertical flips and 90° rotations are applied **identically** to T_1 , T_2 and the mask to preserve spatial correspondence, while brightness, contrast and colour are jittered

independently for each date (contrast 0.8–1.2, brightness ± 20 , per-channel shift ± 8) so that the network learns that illumination differences are not change (Figure 3).

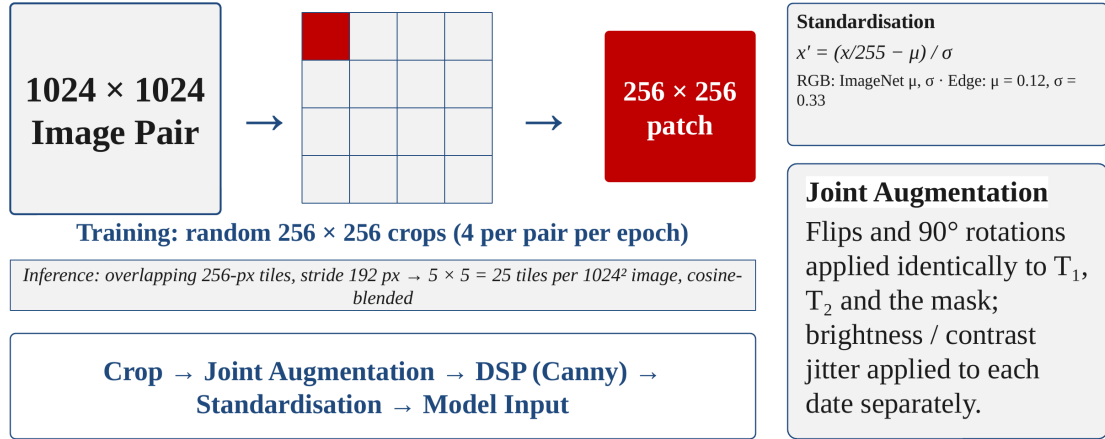


Figure 3: Preprocessing: native-resolution cropping for training, overlapping tiles for inference, joint augmentation and channel standardisation.

3.2.2 DSP Front-End

Each RGB image passes through a classical edge-extraction chain (Figure 4) [11]. First, luminance is computed with the ITU-R BT.601 weights [12]:

$$Y(x,y) = 0.299 R(x,y) + 0.587 G(x,y) + 0.114 B(x,y) \quad (2)$$

The grayscale image is then smoothed by 2-D convolution with a 5×5 Gaussian kernel of $\sigma = 1$, a low-pass filter that removes high-frequency sensor noise which would otherwise produce spurious gradients [7]:

$$G(x,y) = (1 / 2\pi\sigma^2) \cdot \exp(-(x^2 + y^2) / 2\sigma^2) \quad (3)$$

$$I_G(x,y) = (G * Y)(x,y) = \sum_m \sum_n G(m,n) \cdot Y(x-m, y-n) \quad (4)$$

The Canny detector [8] estimates the gradient with Sobel kernels, computes its magnitude and direction, thins edges by non-maximum suppression along the gradient direction, and applies hysteresis thresholding with a low threshold of 50 and a high threshold of 150 (pixels above 150 are strong edges; pixels between 50 and 150 are kept only if connected to a strong edge):

$$G_x = S_x * I_G, \quad G_y = S_y * I_G, \quad S_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad S_y = S_x^T \quad (5)$$

$$M(x,y) = \sqrt{G_x^2 + G_y^2}, \quad \boxed{M(x,y) = |G_x| + |G_y|}, \quad \theta(x,y) = \text{atan2}(G_y, G_x) \quad (6)$$

The binary edge map $E \in \{0,1\}$ is appended to the RGB image as a fourth channel, and every channel is standardised to zero mean and unit variance (ImageNet statistics for RGB; $\mu = 0.12, \sigma = 0.33$ for the edge channel, see Section 3.1.4):

$$x'_c = (x_c - \mu_c) / \sigma_c, \quad c \in \{R, G, B, E\} \quad (7)$$

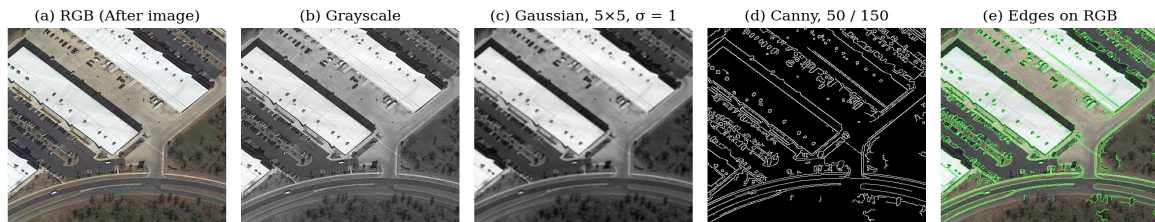


Figure 4: DSP front-end on a LEVIR-CD crop: (a) RGB, (b) grayscale, (c) Gaussian-filtered, (d) Canny edge map, (e) edges overlaid on the RGB image.

Frequency-domain view. Frequency-domain view. The Gaussian smoothing and Sobel derivative stages are linear shift-invariant 2-D filters, so their spatial-frequency responses can be analyzed directly. The finite 5×5 Gaussian kernels have a Gaussian-like low-pass response, while the Sobel operator emphasizes local spatial derivatives and attenuates the DC component. The Gaussian smoothing followed by derivative filtering forms a derivative-of-Gaussian-type gradient stage; the complete Canny detector then applies nonlinear non-maximum suppression and hysteresis. (Figure 5). The 2-D DFT of the finite 5×5 Gaussian kernels have a Gaussian-like low-pass magnitude response. it passes the low spatial frequencies that carry building shapes, falls to -3 dB at about 0.27π rad/sample and attenuates the highest frequencies (sensor noise, texture) to about 2 % at $\omega = \pi$. The Sobel kernel $S_x = [1 \ 2 \ 1]^T[-1 \ 0 \ 1]$ combines a first-difference in ω_1 , $|H| \propto |\sin \omega_1|$, with smoothing in ω_2 , giving a band-pass response that peaks at $\omega_1 = \pm\pi/2$ and is zero at DC: it responds to intensity edges but not to flat regions. Cascading the two (Gaussian, then Sobel inside Canny) is the classical derivative-of-Gaussian edge detector [7], [8].

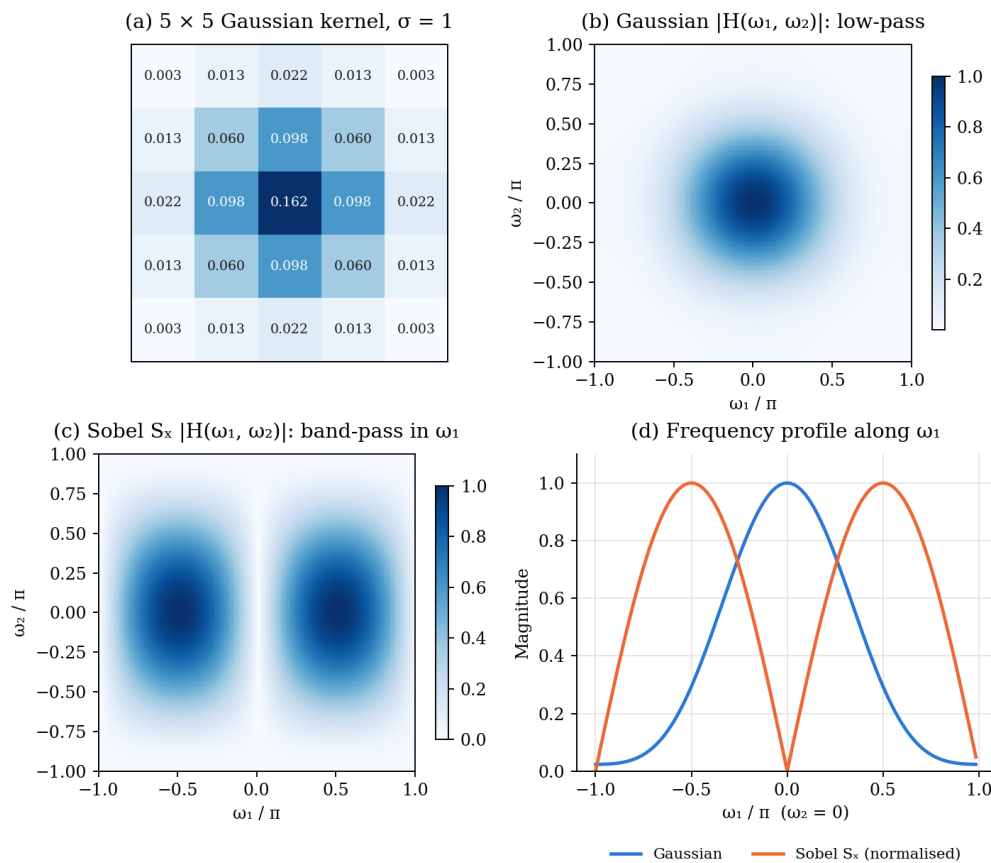


Figure 5: DSP filters in the frequency domain: (a) the 5×5 Gaussian kernel ($\sigma = 1$); (b) magnitude of its 2-D DFT, a low-pass filter; (c) magnitude response of the Sobel kernel S_x , a band-pass (derivative) filter; (d) both responses along ω_1 at $\omega_2 = 0$.

3.2.3 Siamese U-Net Architecture

Figure 6 shows the network (26.0 million parameters). Both four-channel inputs pass through the **same** ResNet-34 encoder [13] pre-trained on ImageNet (shared weights, i.e. a Siamese encoder), producing features at five scales (1/2 to 1/32 of the input, 64 to 512 channels). To accept four channels without discarding the pre-trained weights, the first convolution is widened

$$W_{\text{new}}[:, :, 3, :, :] = W_{\text{old}}, \quad W_{\text{new}}[:, :, 4, :, :] = \frac{1}{3} \sum_{c=0}^2 W_{\text{old}}[:, :, c, :, c+1:] \quad (8)$$

The pretrained RGB filters are copied unchanged to the first three input channels, while the fourth-channel weights are initialized as the mean of the pretrained RGB filters. At every scale the two feature maps are fused (Section 3.2.4), and a U-Net decoder [14] with skip connections up-samples the fused features from 1/32 to 1/2 resolution, followed by a $\times 2$ up-sampling and a 3×3 full-resolution refinement layer. An auxiliary **edge head** receives the fused 1/2- and 1/4-scale features plus global context from the bottleneck and predicts the outlines of changed buildings; its features are concatenated into the last decoder stage (edge-guided fusion).

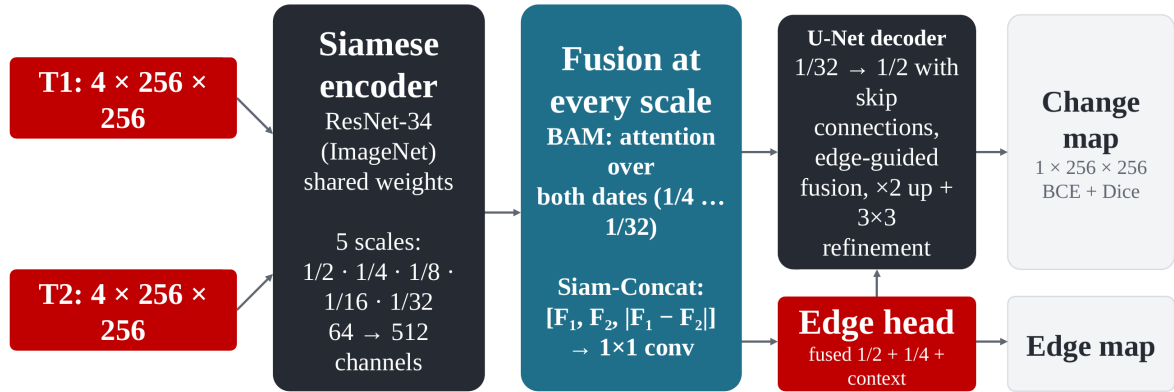


Figure 6: Siamese U-Net with attention-based fusion and an edge-guided decoder.

3.2.4 Spatial-Temporal Attention and Siam-Concat Fusion

The basic spatial-temporal attention module (BAM) of STANet [3] is applied at scales 1/4 to 1/32. The two feature maps $F_1, F_2 \in \mathbb{R}^{C \times H \times W}$ are placed side by side, $X = [F_1 \parallel F_2] \in \mathbb{R}^{C \times H \times 2W}$, so every pixel of either date can attend to every pixel of both dates. With 1×1 convolutions producing queries Q , keys K and values V (keys and values average-pooled by $r = 4$ or 2 at high-resolution scales), scaled dot-product attention [15] gives

$$A = \text{softmax}(Q^T K / \sqrt{d}), \quad X' = X + \gamma \cdot V A^T \quad (9)$$

where $d = C/8$ is the key dimension and γ is a learnable scalar initialised to 0, so that the module starts as an identity mapping and does not disturb the pre-trained features. The attended features are then fused by concatenating both dates and their absolute difference (Siam-Concat) and compressing with a 1×1 convolution, batch normalisation and ReLU:

$$D = |F'_1 - F'_2|, \quad F = \text{ReLU}(\text{BN}(W_{1 \times 1} * [F'_1, F'_2, D])) \quad (10)$$

3.2.5 Loss Function

The network is trained with a multi-task loss combining binary cross-entropy (BCE), Dice loss [16] and a class-balanced edge loss [9]:

$$L = L_{\text{BCE}} + L_{\text{Dice}} + 0.4 \cdot L_{\text{edge}} \quad (11)$$

$$L_{\text{BCE}} = -(1/N) \sum_i [y_i \log p_i + (1 - y_i) \log (1 - p_i)] \quad (12)$$

$$L_{\text{Dice}} = 1 - (2 \sum_i p_i y_i + 1) / (\sum_i p_i + \sum_i y_i + 1) \quad (13)$$

$$\beta = \frac{|E^c|}{|\Omega|}, \quad w_i = \begin{cases} \beta, & E_i = 1 \\ 1 - \beta, & E_i = 0 \end{cases} \quad (14)$$

where:

- E = edge pixels
- E^c = non-edge pixels
- Ω = all pixels

BCE penalises each pixel independently, whereas the Dice term measures the overlap of the changed region directly and is therefore insensitive to the 95 % unchanged background. The edge target is obtained by applying the Sobel operator to the ground-truth mask, $E = 1[\sqrt{((S_x * Y)^2 + (S_y * Y)^2)} > 0.5]$. Because boundary pixels are only 1–2 % of all pixels, the edge BCE is weighted by β (the fraction of non-edge pixels) for edge pixels and $1 - \beta$ for non-edge pixels.

3.2.6 Training Procedure

Table 3 lists the training settings. The learning rate follows a one-epoch linear warm-up and a cosine decay stepped every iteration:

$$\eta(k) = \eta_0 \cdot [\rho + (1 - \rho) \cdot \frac{1}{2} (1 + \cos(\pi(k - k_w) / (K - k_w)))], \quad k \geq k_w \quad (15)$$

where $\eta_0 = 10^{-4}$, $\rho = 0.01$, k_w is the number of warm-up iterations and K the total number of iterations. After every epoch the model predicts the full-resolution validation images; the checkpoint with the highest validation F1 (over a threshold sweep) is kept, and training stops if validation F1 does not improve for 15 epochs.

Table 3: Training settings.

Setting	Value
Input	$4 \times 256 \times 256$ crops (RGB + Canny), 4 crops per image per epoch
Batch size / iterations per epoch	4 / 445
Optimiser	AdamW [17], learning rate 1×10^{-4} , weight decay 1×10^{-4}
Schedule	1-epoch warm-up + cosine decay; gradient clipping at norm 1.0
Loss weights	BCE 1.0, Dice 1.0, edge 0.4
Maximum epochs / early stopping	50 / patience 15 on validation F1
Hardware	Apple-silicon MacBook GPU (PyTorch MPS) & Google Colab on windows PC, ≈ 6.3 min per epoch
Result	best validation F1 0.9107 at epoch 32; stopped at epoch 47

3.2.7 Inference, Thresholding and Post-Processing

A full scene is predicted with overlapping 256×256 tiles (stride 192 px, i.e. 25 tiles for a 1024×1024 image). Tile logits are blended with a separable raised-cosine window w that tapers over the 64-pixel overlap, which removes seams and down-weights tile borders that see less context:

$$z(x,y) = \sum_k w_k(x,y) z_k(x,y) / \sum_k w_k(x,y) \quad (16)$$

Test-time augmentation (TTA) averages the probabilities of the identity, horizontal-flip, vertical-flip and 180°-rotation views, each mapped back to the original orientation. A pixel is labelled *changed* when its probability is at least $t = 0.41$, the threshold that maximised F1 on the validation set. In the GUI the mask can optionally be cleaned by a 3×3 morphological opening, removal of regions smaller than 20 pixels, and Douglas–Peucker polygon simplification [18] with a tolerance of 1 % of each contour’s perimeter; an image-specific Otsu threshold [19] is also offered.

3.2.8 Evaluation Metrics

With TP, FP, FN and TN counted over **all pixels of all test images** (dataset-level, the convention of the literature), the metrics are:

$$P = TP / (TP + FP), \quad R = TP / (TP + FN), \quad F1 = 2PR / (P + R) \quad (17)$$

$$IoU = TP / (TP + FP + FN) = F1 / (2 - F1) \quad (18)$$

$$OA = (TP + TN) / (TP + TN + FP + FN) \quad (19)$$

Precision (P) measures the fraction of pixels predicted as changed that are actually changed, i.e., how reliably the model’s positive predictions identify true change. **Recall (R)** measures the fraction of all truly changed pixels that are successfully detected, i.e., how much actual change is recovered by the model. F1 is the harmonic mean of precision and recall and therefore balances false alarms (FP) against missed changes (FN).

IoU (Intersection over Union) measures the overlap between the predicted change region and the ground-truth change region. For binary segmentation, IoU and F1 are mathematically related through $IoU = F1 / (2 - F1)$.

Overall Accuracy (OA) measures the fraction of all pixels classified correctly, including both changed and unchanged pixels. OA is reported for completeness but is not used for design decisions because the dataset is highly imbalanced: approximately 95% of pixels are unchanged, so a model predicting "no change" for every pixel could still obtain approximately 95% OA while detecting no actual changes.

Cohen’s κ is also reported as a chance-corrected agreement measure. Unlike OA, κ accounts for the agreement that could be expected from the class distribution by chance.

For interpretation, F1, IoU, precision and recall are the primary change-detection metrics, while OA and Cohen’s κ provide supplementary measures of overall classification and chance-corrected agreement, respectively.

3.3 Source Code (Core Modules)

The complete source code (about 30 Python modules: data pipeline, DSP front-end, model, losses, training, evaluation, analysis scripts and GUI) is available in the project repository (Section 7.3). Table 4 lists the core DSP and model routines.

Table 4: Core source code: DSP front-end, 4-channel encoder, attention, fusion, loss and sliding-window inference (excerpts).

```
# preprocessing/dsp_processing.py (DSP front-end)
def rgb_to_grayscale(image):
    # Y = 0.299 R + 0.587 G + 0.114 B (ITU-R BT.601)
    return cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)

def apply_gaussian_filter(gray, kernel_size=5, sigma=1.0):
    # 2-D low-pass convolution with a Gaussian kernel
    return cv2.GaussianBlur(gray, (kernel_size, kernel_size),
                             sigmaX=sigma)
```

```

def detect_canny_edges(blurred, low=50, high=150):
    # Sobel gradients -> non-max suppression -> hysteresis
    return cv2.Canny(blurred, threshold1=low, threshold2=high)

def compute_edge_map(rgb_image):
    gray = rgb_to_grayscale(rgb_image)
    blurred = apply_gaussian_filter(gray)
    edges = detect_canny_edges(blurred)
    return edges.astype(np.float32) / 255.0    # {0, 1}

def prepare_model_input(rgb_image, use_canny=True):
    # RGB: ImageNet standardisation
    rgb = rgb_image.astype(np.float32) / 255.0
    rgb = (rgb - IMAGENET_MEAN) / IMAGENET_STD
    if not use_canny:
        return rgb    # H x W x 3
    # Edge: standardised with measured statistics
    edge = (compute_edge_map(rgb_image) - 0.12) / 0.33
    return np.concatenate([rgb, edge[... , None]], -1) # H x W x 4

# models/encoder.py (4-channel ImageNet encoder)
new_conv.weight[:, :3] = old_conv.weight    # RGB filters
new_conv.weight[:, 3:] = old_conv.weight.mean(1, keepdim=True)

# models/fusion.py (spatial-temporal attention, BAM)
class BAM(nn.Module):
    def forward(self, before, after):
        B, C, H, W = before.shape
        x = torch.cat([before, after], dim=3)    # dates side by side
        x_kv = torch.cat([self.pool(before),
                          self.pool(after)], dim=3)
        q = self.q_conv(x).flatten(2)    # [B, d, N]
        k = self.k_conv(x_kv).flatten(2)    # [B, d, M]
        v = self.v_conv(x_kv).flatten(2)    # [B, C, M]
        a = torch.bmm(q.transpose(1, 2), k) * self.key_channels ** -0.5
        a = a.softmax(dim=-1)    # [B, N, M]
        out = torch.bmm(v, a.transpose(1, 2)).view(B, C, H, 2 * W)
        out = x + self.gamma * out    # gamma = 0 at start
        return out[... , :W], out[... , W:]

# Siam-Concat fusion at every scale
d = torch.abs(bf - af)
fused = reduce_conv(torch.cat([bf, af, d], dim=1))    # 3C -> C

# losses/losses.py (multi-task loss)
bce = F.binary_cross_entropy_with_logits(change_logits, mask)
dice = 1 - (2 * (p * y).sum() + 1) / (p.sum() + y.sum() + 1)
edge_target = sobel_edge_detection(mask)    # |grad| > 0.5
beta = (edge_target == 0).float().mean()
w = beta * edge_target + (1 - beta) * (1 - edge_target)
edge = F.binary_cross_entropy_with_logits(edge_logits, edge_target,
                                          weight=w)
loss = bce + dice + 0.4 * edge

# models/siamese_unet.py (sliding-window inference)
for (y, x) in tile_positions(H, W, tile=256, stride=192):
    out = model(before[... , y:y+256, x:x+256],
                after[... , y:y+256, x:x+256])
    acc[... , y:y+256, x:x+256] += out * cosine_window
    wsum[... , y:y+256, x:x+256] += cosine_window
logits = acc / wsum

```

4 Implementation

4.1 Description

The system is implemented in Python 3 with PyTorch [20] for the neural network, OpenCV

[21] for the DSP operations and PyQt5 for the user interface. Table 5 gives the software organisation.

Table 5: Software modules.

Module	Function
config.py	All settings: paths, DSP parameters, normalisation statistics, training hyper-parameters, threshold
preprocessing/	Grayscale, Gaussian and Canny DSP chain; channel standardisation
dataset/	LEVIR-CD loader, random / change-focused cropping, synchronised and photometric augmentation
models/	Siamese ResNet-34 encoder, BAM + Siam-Concat fusion, U-Net decoder, edge head, sliding-window inference
losses/, training/	Multi-task loss, training loop, dataset-level metrics, threshold sweep
train.py, evaluate.py	Training (with resume and ablation flags) and full-resolution test evaluation with reports
inference/, gui/app.py	Prediction on any image pair, post-processing and the PyQt5 desktop application
inference/multi_model.py	One interface for v2 (4-ch), v2 RGB-only, the old v1 model and two v1 + v2 ensembles
compare_models.py, save_all_samples.py	Cross-dataset comparison (LEVIR-CD, WHU-CD, Purbachal) and per-image comparison figures
sanity_check.py	16 automated checks of shapes, normalisation, batch independence, loss and checkpoints

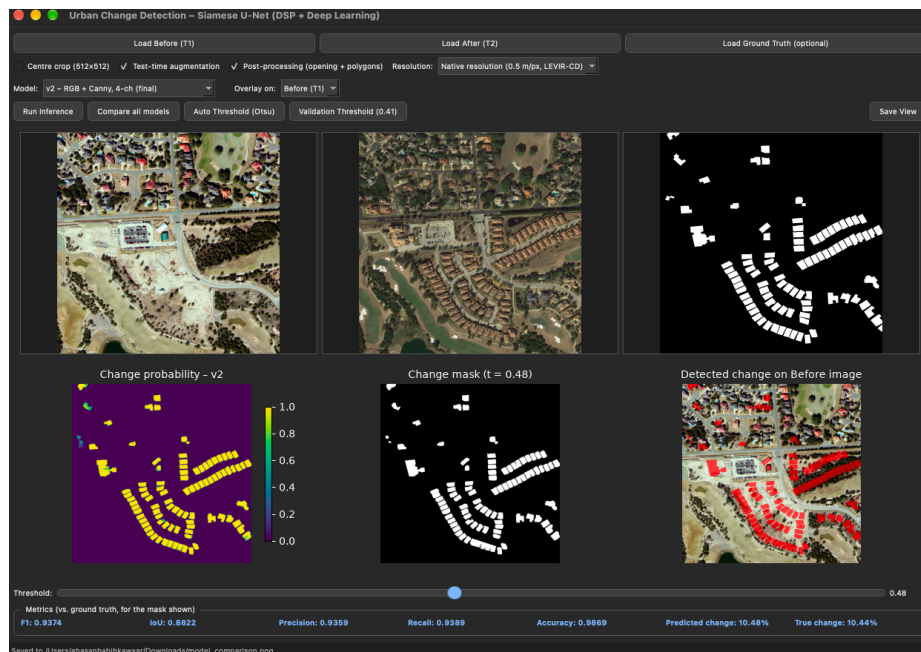


Figure 7: PyQt5 desktop application with the model selector, overlay option and live metrics.

Graphical User Interface: The desktop application (Figure 7) loads the before and after

images and, optionally, a ground-truth mask. The model is loaded once in a background thread and inference runs in a separate QThread, so the window never freezes. A **Model** menu selects the final v2 model (RGB + Canny), the RGB-only v2 model, the old v1 prototype or two v1 + v2 ensembles (average and pixel-wise maximum), each with its own validation-selected threshold, and **compare all models** shows every model side by side on the same image pair. The user can enable test-time augmentation, choose the input resolution, move a threshold slider (or use the validation or Otsu threshold), switch polygon post-processing on or off and draw the red change overlay on the Before (default) or After image; the probability map, the mask and live F1/IoU/precision/recall values update instantly. In Figure 7 the final model reaches $F1 = 0.937$ and $IoU = 0.882$ on a LEVIR-CD pair at threshold 0.48, and Figure 8 shows the comparison window for the same pair: the v2 models give the sharpest building outlines, while the old v1 model produces blurred, merged blobs ($F1\ 0.80$).

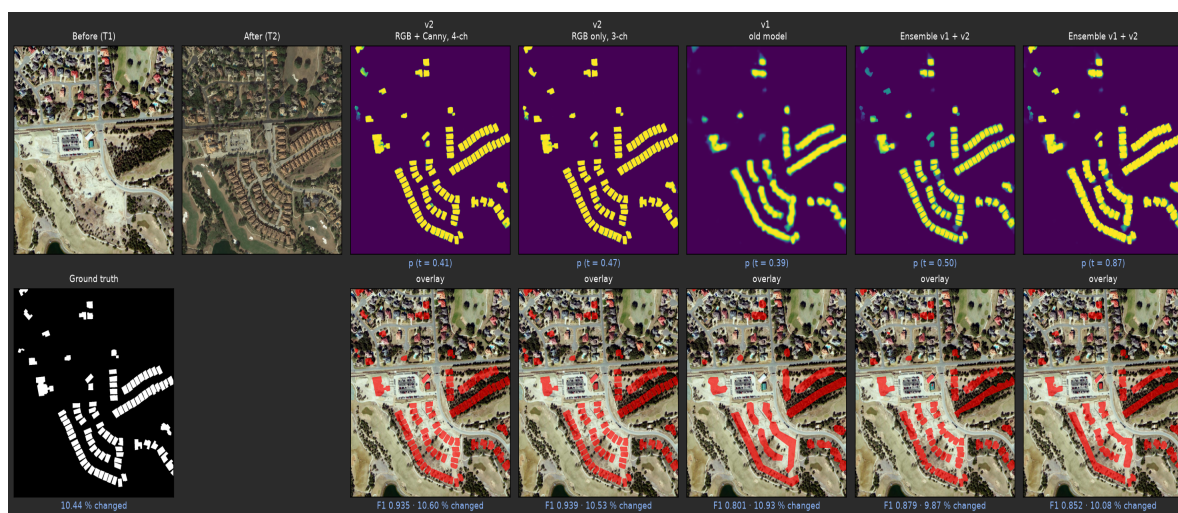


Figure 8: “Compare all models” window on the same pair: probability maps (top) and change overlays on the Before image with per-model F1 (bottom) for v2 4-channel, v2 3-channel, the old v1 model and the two ensembles.

Qualitative results. Figure 9 shows three test scenes: a warehouse development, a new housing estate and a curved suburban layout. The error maps are almost entirely white (correct), with red (false alarm) and cyan (miss) limited to 1–2-pixel rims along building outlines; closely spaced houses are separate

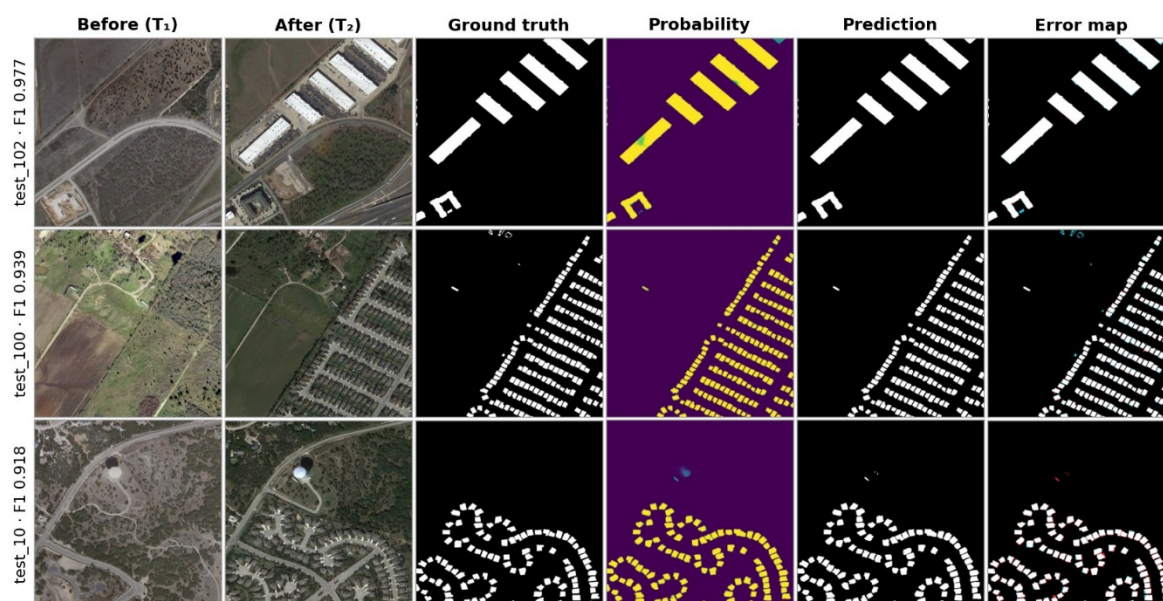


Figure 9: Final model on three LEVIR-CD test scenes: test_102 (F1 0.977, IoU 0.956), test_100 (F1 0.939, IoU 0.885) and test_10 (F1 0.918, IoU 0.848). Error map: white = correct change, red = false alarm, cyan = miss.

Bangladeshi imagery: To test generalization, all models were applied to 12 bi-temporal 1024×1024 image pairs of Purbachal, Dhaka, in which open plots became dense apartment blocks (Figure 10). No ground truth exists for these pairs. The old v1 model marks far more pixels as changed than the final model, and many of its marks fall on plots that are built up in T_2 ; the final 4-channel model is very conservative. Section 5.3.3 (E10) quantifies this and explains why it cannot be scored.

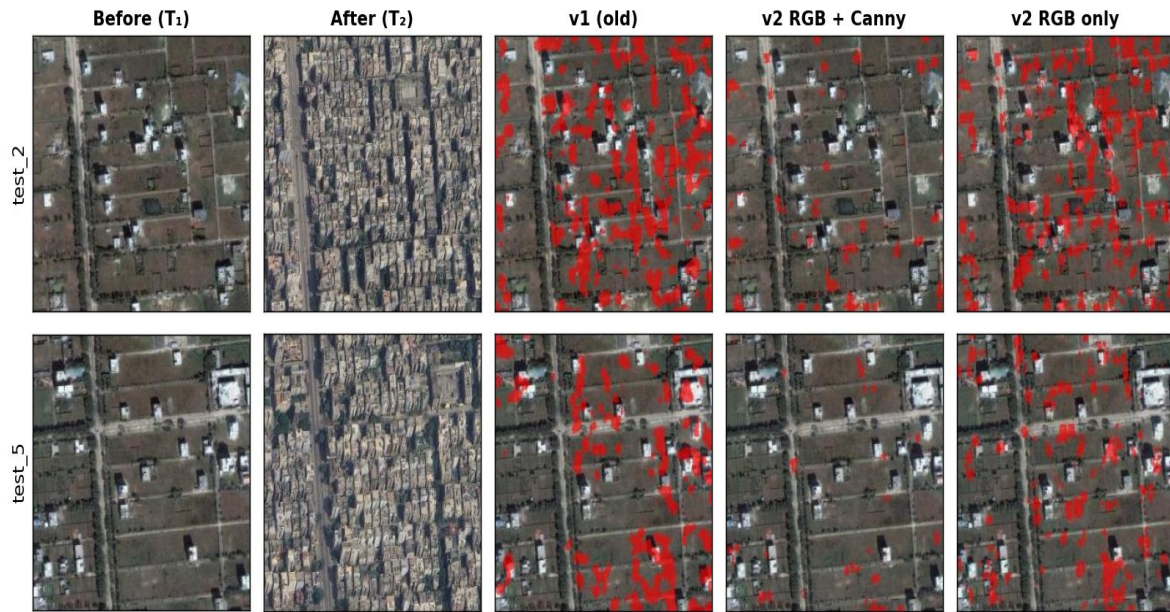


Figure 10: Purbachal, Dhaka (test_2, test_5): predicted change (red) of the old v1 model and the v2 models, drawn on the Before image. No ground truth is available.

5 Design Analysis and Evaluation

5.1 Novelty

- **Hybrid DSP–deep-learning input:** a classical Gaussian/Canny edge map, standardized with measured statistics, is fused with RGB inside an ImageNet-pre-trained encoder whose first layer is analytically extended to four channels.
- **Attention and Siam-Concat fusion at every scale,** with spatial-reduction attention that makes spatial-temporal attention affordable on a laptop GPU.
- **Edge-guided decoding:** a DSP operator (Sobel) generates the supervision for an auxiliary boundary head whose features steer the change decoder.
- **Native-resolution deployment:** cosine-blended sliding-window inference, a validation-selected threshold and a real-time desktop application for non-experts.
- **Cross-dataset, multi-model evaluation:** the old and new models, the 3- and 4-channel inputs and two ensembles are compared under one protocol on LEVIR-CD, an unseen labelled city (WHU-CD) and Bangladeshi imagery, and all are selectable in the GUI.

- **Engineering investigation:** three silent defects of the first prototype were diagnosed with controlled tests and fixed, and the contribution of the DSP channel was measured by ablation rather than assumed.

5.2 Design Considerations (PO(c))

5.2.1 Considerations to public health and safety

- The system can flag unauthorized construction in flood-flow zones, on wetlands and in earthquake-sensitive areas of Dhaka for inspection, and can rapidly map damaged or new structures after floods and cyclones.
- A missed building (false negative) could hide a hazard, so the tool is designed as **decision support**: results are shown with probabilities and overlays for a human to verify, and the threshold slider lets the user favor recall when safety matters.
- Precision and recall were balanced (91.8 % and 90.6 %) rather than maximizing one at the expense of the other.

5.2.2 Considerations to environment

- Monitoring encroachment on wetlands, agricultural land and green cover supports environmental protection.
- Remote analysis replaces repeated field-survey trips, reducing travel and fuel use.
- The model was deliberately kept lightweight (26 M parameters) and trained on a laptop in about five hours, a small energy footprint compared with large transformer models; the whole system runs offline.

5.2.3 Considerations to cultural and societal needs

- Up-to-date change maps help planning bodies such as RAJUK and city corporations make informed decisions.
- The GUI requires no programming knowledge, produces polygons usable in GIS software, and reports the changed-area percentage in plain terms.
- Informal settlements are home to many low-income families; the report and user manual state that the tool must not be used to target such communities for eviction.

5.3 Investigations (PO(d))

5.3.1 Design of Experiment

The following controlled experiments were designed. Unless stated otherwise, all use the same 128 LEVIR-CD test images, dataset-level metrics and the validation-selected threshold 0.41; only one factor is changed at a time. E8–E10 use the same protocol for every model: full-resolution prediction and a threshold chosen per model on LEVIR-CD validation.

Table 6: Designed experiments.

ID	Experiment	Factor varied	Purpose
E1	Final evaluation	–	Performance of the final RGB + Canny model with TTA
E2	TTA ablation	TTA on / off	Effect of test-time augmentation
E3	DSP ablation	Canny channel on / off (retrained)	Does the explicit edge channel help?
E4	Prototype comparison	v1 (resized, defective) vs v2	Effect of the design corrections
E5	Threshold sensitivity	t from 0.05 to 0.95	Robustness of the chosen threshold
E6	Batch-independence test	Batch composition	Verify the attention module does not mix samples
E7	Error analysis	Amount of change per image	Where and why the model fails
E8	Cross-dataset test	LEVIR-CD vs WHU-CD (unseen city)	Do v1, v2 4-ch and v2 3-ch generalise?
E9	Model ensembles	v1 + v2 average / maximum	Can the old and new model be combined?
E10	Bangladeshi imagery	Model; input scale $\times 1$, $\times 2$, $\times 4$	Behaviour on Purbachal pairs (no labels)

5.3.2 Data Collection

- For every experiment, each of the 128 test images ($1024 \times 1024 = 1,048,576$ pixels) was predicted at full resolution, giving 134,217,728 labelled pixels in total.
- The confusion counts TP, FP, FN, TN were accumulated over all pixels. Probabilities were binned into 1,000 bins separately for changed and unchanged pixels, so precision, recall, F1 and IoU could be computed exactly for any threshold from one pass.
- Per-image metrics, the ground-truth changed-area percentage and the false-positive / false-negative pixel counts were saved to CSV files for the error analysis.
- Training and validation losses, validation F1 and the best validation threshold were logged every epoch for both the RGB + Canny and the RGB-only models.
- For E8–E10, compare_models.py ran every model on LEVIR-CD validation (thresholds), LEVIR-CD test (128 pairs, TTA), WHU-CD test (638 labelled 512×512 tiles, v2 input resampled $\times 0.6$ to 0.5 m/pixel, no TTA) and the 12 Purbachal pairs (predicted-change area only), and saved per-image CSV files and comparison figures.

5.3.3 Results and Analysis

E1 – Final evaluation. Table 7 and Figure 11 give the final test results.

Table 7: Final test-set results (EI, RGB + Canny, TTA, threshold 0.41).

Metric	Value	Metric	Value
Precision	91.76 %	True positives	6,194,312
Recall	90.59 %	False positives	556,557
F1-score	91.17 %	False negatives	643,092
IoU	83.78 %	True negatives	126,823,767
Overall accuracy	99.11 %	Cohen's κ	0.907

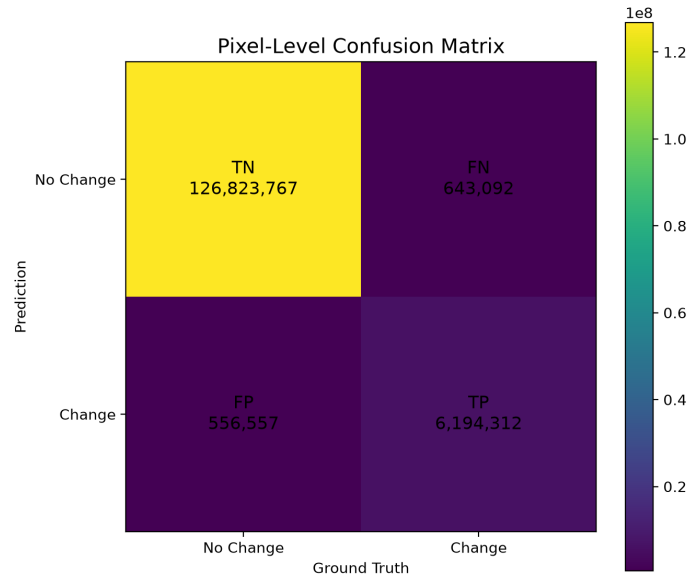


Figure 11: Pixel-level confusion matrix on the test set.

As a consistency check, substituting the measured counts into (16) gives $P = 6,194,312 / 6,750,869 = 0.9176$, $R = 6,194,312 / 6,837,404 = 0.9059$, $F1 = 0.9117$ and $IoU = 6,194,312 / 7,393,961 = 0.8378$, and the theoretical relation $IoU = F1 / (2 - F1) = 0.9117 / 1.0883 = 0.8378$ agrees exactly with the measurement.

Learning behaviour. Figure 12 shows that the training loss falls from 1.28 to 0.17 while validation F1 rises quickly to about 0.89 by epoch 9 and saturates near 0.91; the validation loss decreases smoothly without divergence, indicating no harmful over-fitting.

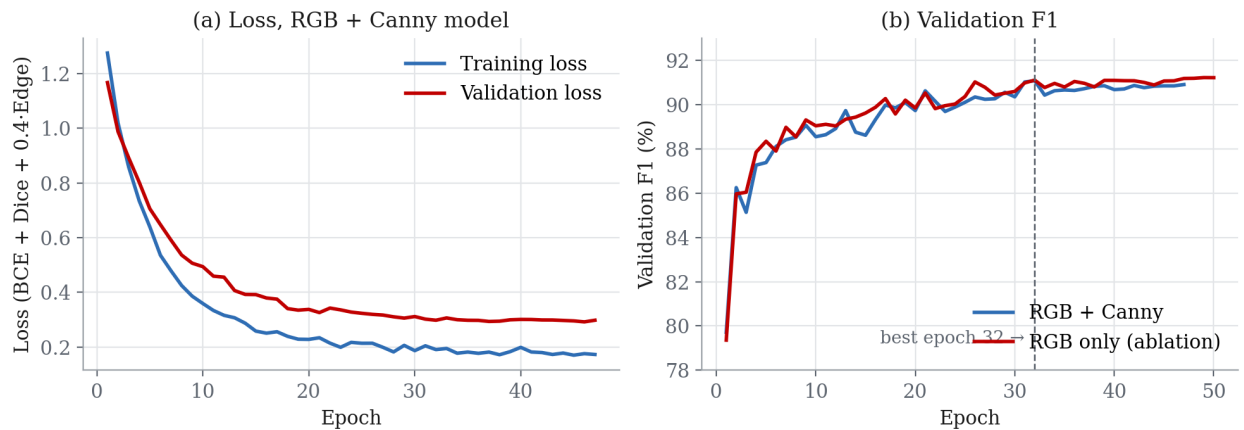


Figure 12: (a) Training and validation loss of the RGB + Canny model; (b) validation F1 of the RGB + Canny and RGB-only models.

E2–E4 – Ablations and prototype comparison. Table 8 and Figure 13 compare the configurations.

Table 8: Ablation results on the 128 test images (threshold 0.41).

Configuration	Input	P (%)	R (%)	F1 (%)	IoU (%)
v1 prototype (256 ² resized, 3 defects)	RGB + Canny	75.78	78.50	77.12	62.76
v2, no TTA (E2)	RGB + Canny	91.83	89.74	90.77	83.10
v2 + TTA, final (E1)	RGB + Canny	91.76	90.59	91.17	83.78
v2 + TTA, DSP ablation (E3)	RGB only	91.78	91.48	91.63	84.56

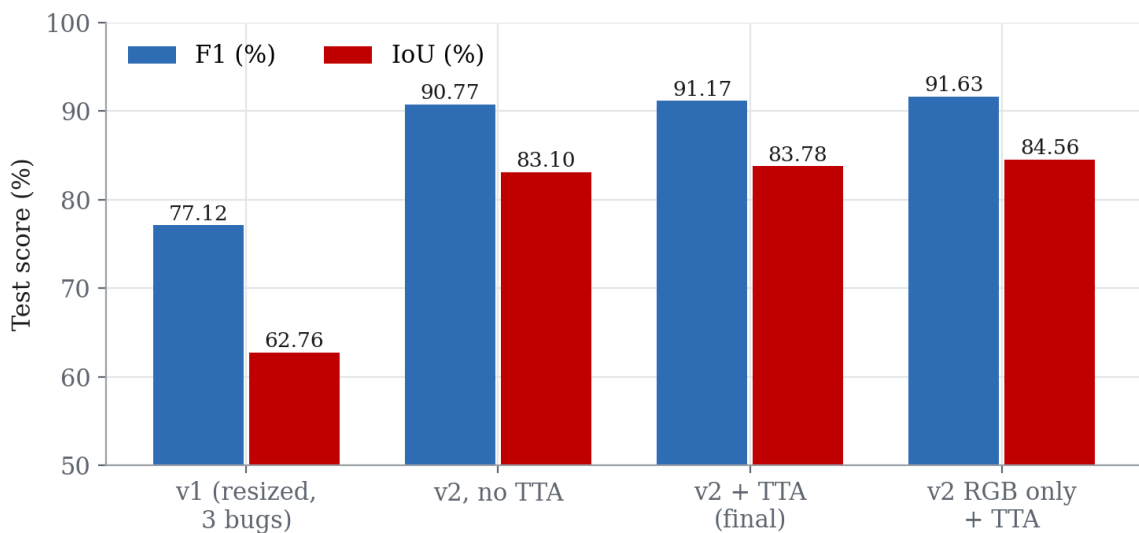


Figure 13: Test F1 and IoU of the four configurations.

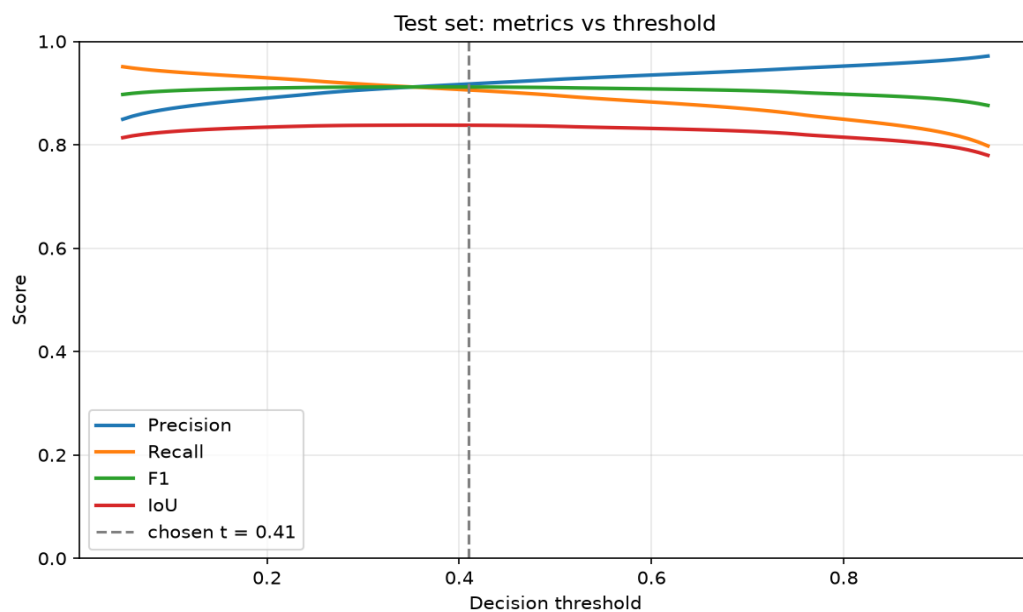


Figure 14: Precision, recall, F1 and IoU on the test set versus the decision threshold.

E5 – Threshold sensitivity. Figure 14 plots the test metrics against the threshold. F1 is flat between about 0.3 and 0.6: the validation-selected threshold (0.41) gives $F1 = 0.9117$, while the best threshold that could be chosen on the test set (0.36) would give only 0.9118, so the result is not over-tuned.

E6 – Batch independence. With the attention output forced active, the first prototype changed a sample’s prediction by up to 0.241 depending on which other images were in the same batch; the corrected module gives a difference of exactly 0, and a sliding-window prediction on a single tile matches the direct prediction to within 3×10^{-8} .

E7 – Error analysis. Figure 15 groups the test images by the share of changed pixels. F1 rises from 84.5 % for images with less than 1 % change to 92.4 % for images with more than 10 % change; all 10 images without any change produced zero false-alarm pixels. Figure 16 shows that most images have less than 0.5 % false-positive and less than 0.5 % false-negative pixels, and that the two error types are balanced overall (0.41 % and 0.48 % of all pixels). The worst image (Figure 17) combines both failure modes: mobile homes only 3–5 pixels wide are missed, and re-roofed buildings are detected as new.

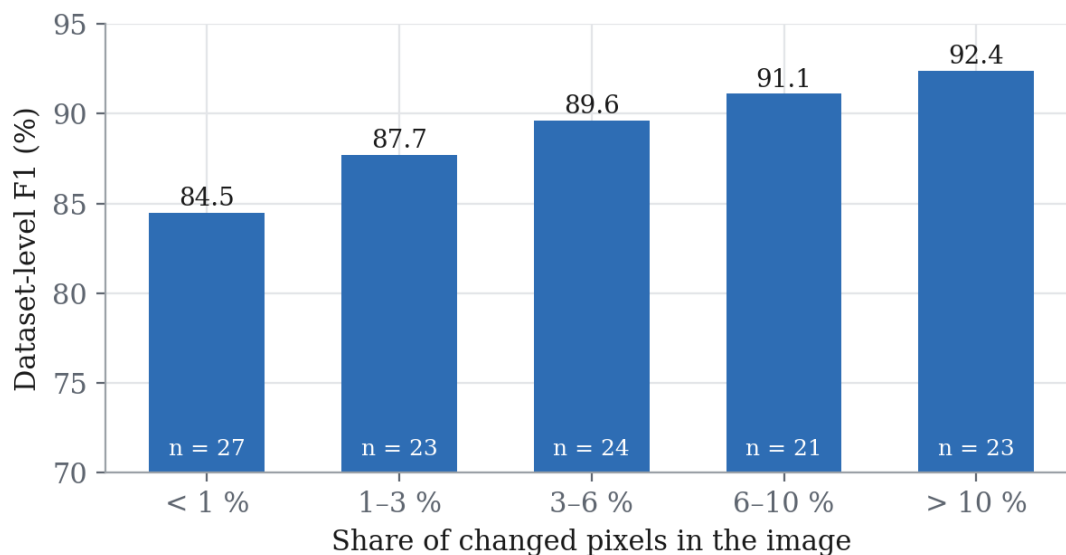


Figure 15: Dataset-level F1 versus the share of changed pixels per image (n = number of test images).

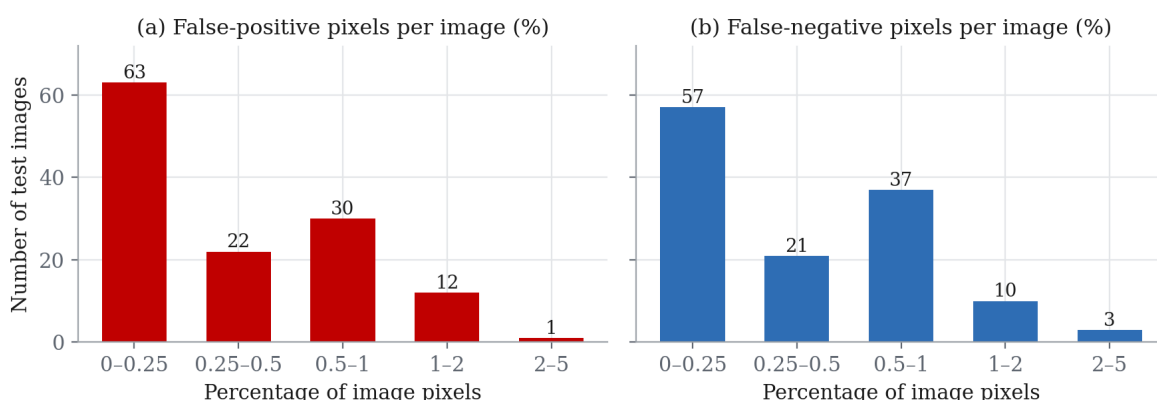


Figure 16: Distribution of false-positive and false-negative pixel percentages over the 128 test images.

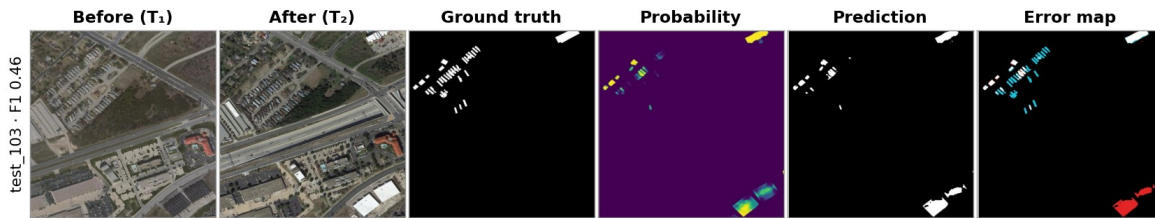


Figure 17: Worst test image, test_103 (F1 0.46): small mobile homes are missed (cyan) and re-roofed buildings produce false alarms (red).

E8–E9 – Old vs new model, 3 vs 4 channels and ensembles. Table 9 and Figure 18 compare all models under one protocol. Note that v1 scores 78.21 % here, slightly higher than the 77.12 % of Table 8, because it is now scored at full resolution with a validation-selected threshold (0.39) instead of on 256×256 masks; the RGB-only model uses its own validation threshold (0.47).

Table 9: Cross-dataset comparison (% , dataset-level). LEVIR-CD: 128 test pairs; WHU-CD: 638 test tiles, never used for training.

Model	Input	LEVIR Precision	LEVIR Recall	LEVIR F1	LEVIR IoU	WHU Precision	WHU Recall	WHU F1	WHU IoU
v1 (old, 256 px)	raw RGB + Canny	75.34	81.30	78.21	64.21	57.33	79.78	66.72	50.06
v2 final	RGB + Canny (4-ch)	91.76	90.59	91.17	83.78	81.62	84.11	82.85	70.72
v2 ablation	RGB only (3-ch)	92.16	91.22	91.69	84.65	80.18	86.21	83.09	71.07
Ensemble average	v1 + v2	90.57	87.73	89.13	80.39	83.18	82.84	83.01	70.95
Ensemble max	v1 + v2	88.14	86.23	87.18	77.27	69.69	85.55	76.81	62.35

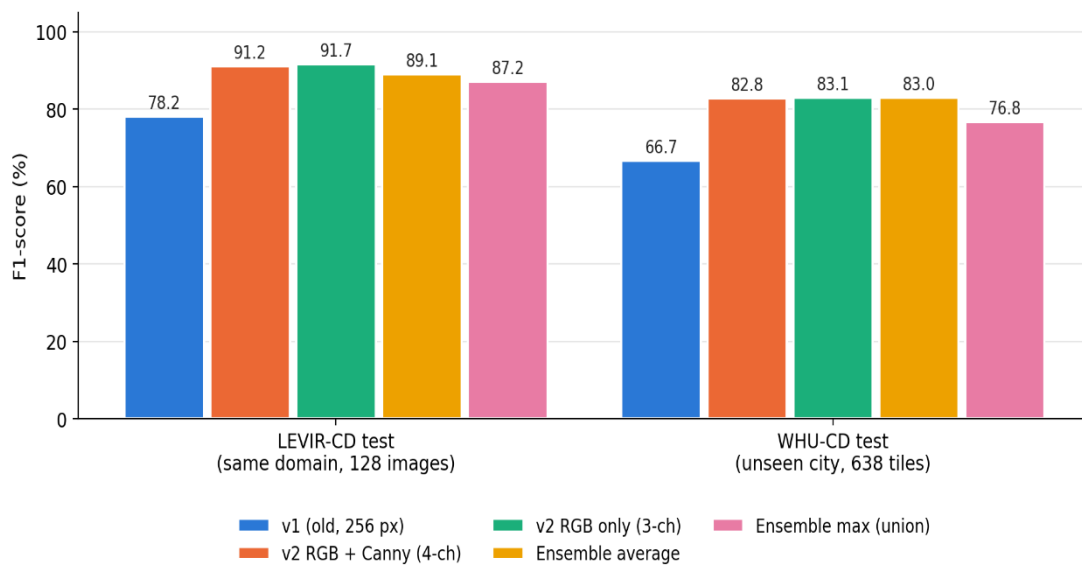


Figure 18: F1-score of the old and new models, the 3- and 4-channel inputs and the two ensembles on LEVIR-CD and WHU-CD.

On WHU-CD the old model loses 11.5 F1 points and its precision falls to 57 %, i.e. it produces many false alarms in an unseen city, while the v2 models lose about 8.5 points and keep precision above 80 %. Resampling WHU-CD from 0.3 to 0.5 m/pixel before v2 inference raised its F1 from 81.05 % to 82.85 %. The RGB-only model is again as good as the 4-channel model (+0.5 F1 on LEVIR-CD, +0.2 on WHU-CD). Averaging v1 and v2 lowers LEVIR-CD F1 by 2.0 points and changes WHU-CD F1 by only +0.2; the pixel-wise maximum adds false alarms on both datasets (Figure 19, Figure 20).

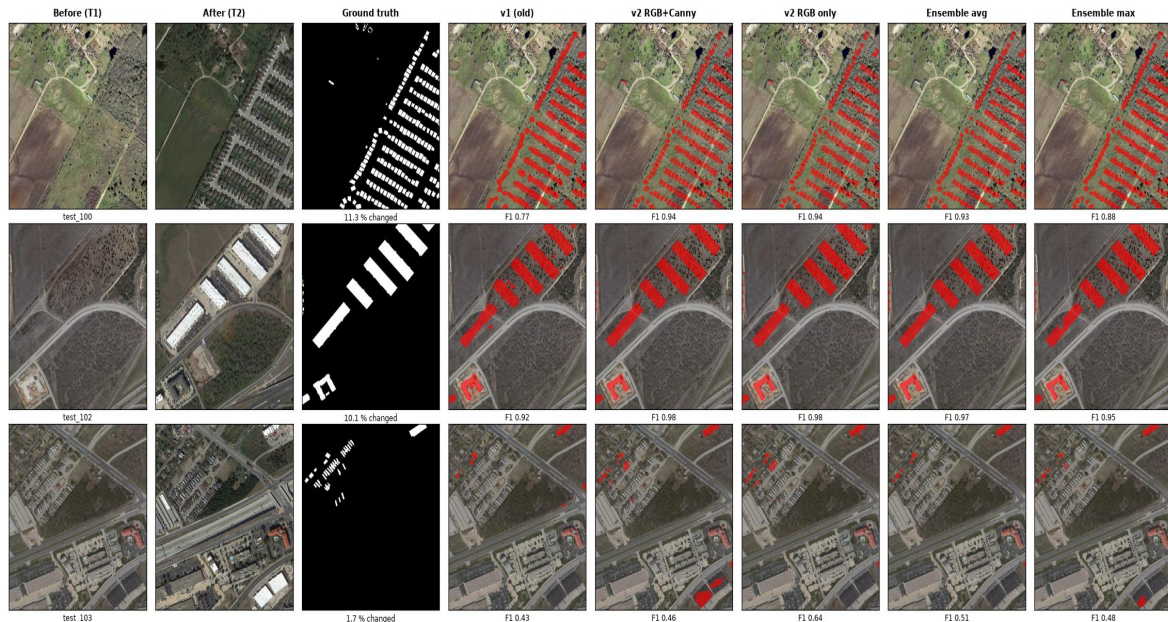


Figure 19: LEVIR-CD test pairs (test_100, test_102, test_103): predicted change (red) of every model on the Before image, with per-image F1. The old v1 model merges neighbouring houses into blobs.

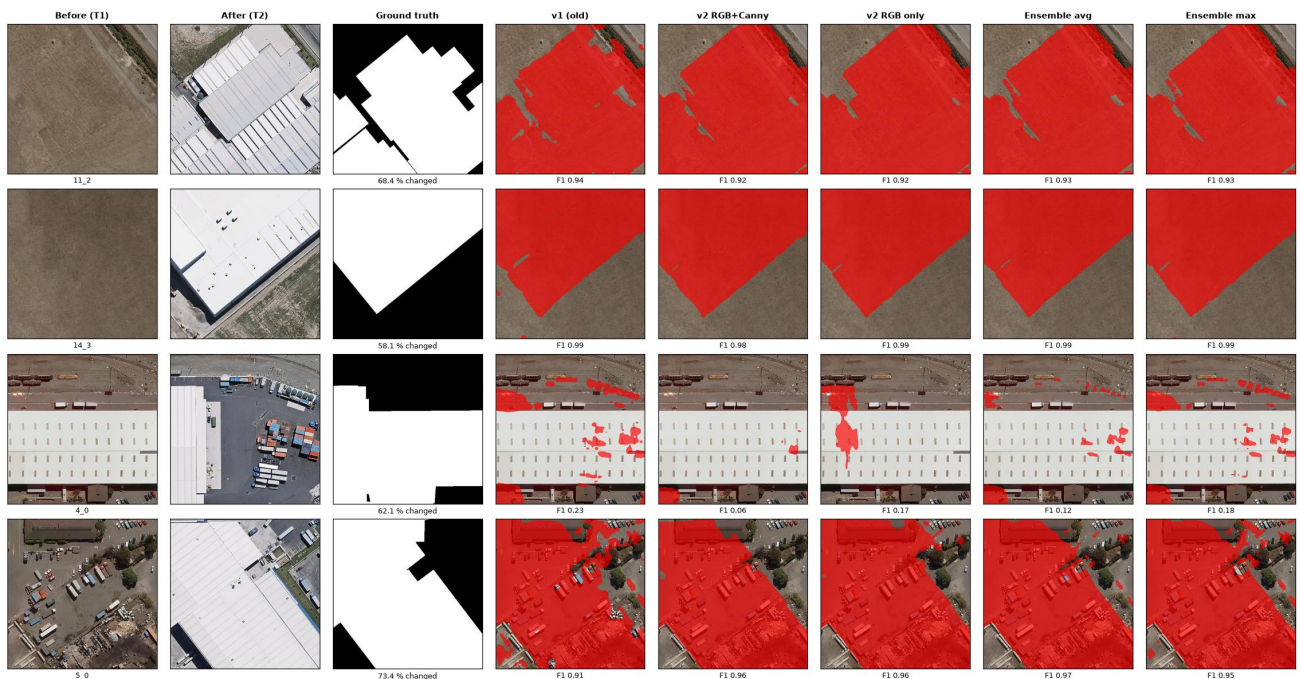


Figure 20: WHU-CD test tiles with the largest changes: predicted change (red) on the Before image, with per-tile F1.

E10 – Bangladeshi imagery: On the 12 Purbachal pairs (Figure 10, Figure 21) the old model

marks on average 9.7 % of the pixels as changed, the RGB-only v2 model 7.9 %, the union ensemble 4.4 %, the final 4-channel model 1.8 % and the averaging ensemble 1.3 %; the v1 and v2 masks overlap little (mean IoU 0.05). Visually, most of the area changed (open plots became dense blocks), so all models under-detect and v1 appears to find the newest construction. On the unlabeled Purbachal scenes, the models produce substantially different changed-area estimates. Visual inspection indicates substantial construction in several pairs, but ground-truth labels are required to determine detection accuracy and quantify missed change. Without ground truth this cannot be scored, and E8 shows that v1's larger detections come with many false alarms on labelled data. Enlarging the input for v2 by $\times 2$ or $\times 4$ reduced the detections further (0.44 % and 0.01 %), so the cause is a domain shift rather than resolution.

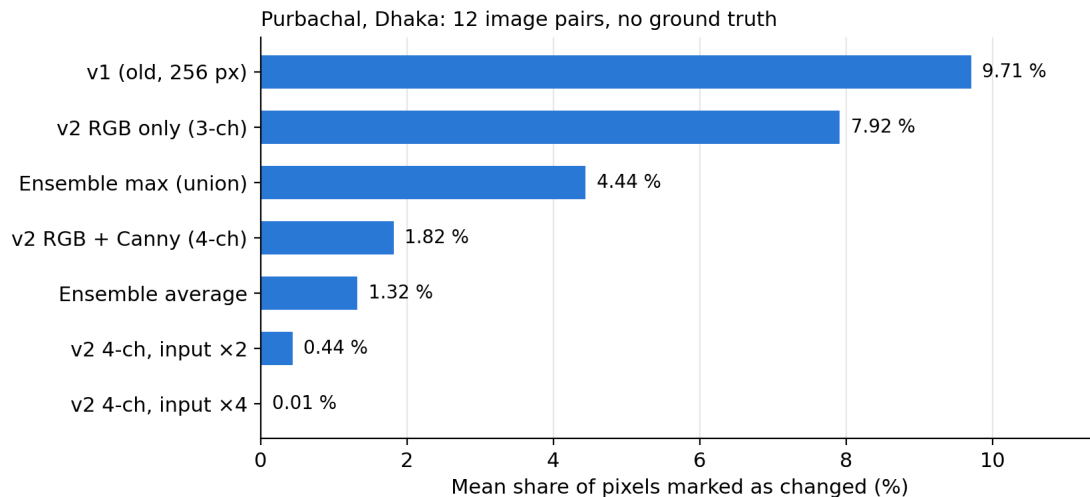


Figure 21: Mean share of pixels marked as changed on the 12 Purbachal pairs (no ground truth).

5.3.4 Interpretation and Conclusions on Data

- The final design meets its objectives: 91.2 % F1 and 83.8 % IoU with balanced errors, competitive with published transformer models while training on a laptop in about five hours and predicting a 1024×1024 scene in about 3 s with TTA.
- The largest gain (+14 F1 points) came from correct engineering of the pipeline, native-resolution processing, correct input standardisation and a batch-independent attention module, rather than from a larger network.
- Test-time augmentation adds a small but consistent improvement (+0.40 F1, +0.68 IoU) at four times the inference cost.
- The fixed Canny input channel did not improve LEVIR-CD test performance in this experiment: F1 changed from 91.17% with RGB + Canny to 91.63% with RGB only, while IoU changed from 83.78% to 84.56%. Because only one training seed was used, the 0.46-point F1 difference is not treated as conclusive evidence that the edge channel is harmful. Instead, the result shows that a fixed Canny channel is not necessary for strong performance in this setup and motivates future experiments with adaptive or learnable edge representations. DSP nevertheless remains central to the overall design through Gaussian/Canny preprocessing, Sobel-based boundary supervision, frequency-domain analysis and signal-aware inference and post-processing.

- The corrected v2 pipeline generalizes substantially better than the first prototype on the unseen WHU-CD dataset. The RGB-only ablation is marginally higher than the 4-channel configuration on both LEVIR-CD and WHU-CD. The RGB + Canny model is retained as the reference configuration because it represents the complete DSP-integrated design, while the RGB-only model serves as the controlled ablation baseline.
- Remaining errors are concentrated on very small objects and on appearance changes of existing roofs. On dense Bangladeshi scenes every model under-detects; the old model reacts more strongly, but only labelled local data can show which is right, so annotating and fine-tuning on Dhaka pairs is the next step.

5.4 Limitations of Tools (PO(e))

Table 10: Limitations of the tools used.

Tool / resource	Limitation	Effect and tolerance
Laptop GPU (Apple MPS)	Shared memory; no float64 support	Batch size limited to 4; ≈ 6.3 min per epoch; metric histograms computed on the CPU
LEVIR-CD dataset	US suburbs only, 0.5 m/pixel, buildings only	No road / vegetation classes; F1 drops by ≈ 8.5 points on WHU-CD; weak transfer to dense Dhaka scenes; some label noise
Bangladeshi imagery	No ground-truth labels	Models can only be compared by predicted area and visual inspection, not by F1
Canny detector (OpenCV)	Fixed thresholds 50 / 150	Edge density depends on image contrast; no measurable accuracy gain on LEVIR-CD
Network input size	256×256 tiles	Context limited to $128 \text{ m} \times 128 \text{ m}$; objects smaller than $\approx 4 \times 4 \text{ px}$ (2 m) are unreliable
Image registration	Pairs assumed co-registered	A shift of a few pixels between dates creates false alarms along edges
Single training run	No repeated seeds	Differences below ≈ 0.5 F1 point (e.g. RGB vs RGB + Canny) are within run-to-run variation
Threshold selection	Chosen on 64 validation images	Optimal test threshold differs (0.36 vs 0.41) but costs only 0.0001 F1

5.5 Ethical Issues (PO(h))

- **Data licence and credit:** LEVIR-CD is used for academic research under the terms of its authors and is not redistributed; every dataset, paper and software library used is cited.
- **Privacy and dual use:** very-high-resolution imagery can reveal private property. The

intended uses are urban planning, environmental monitoring and disaster response, not surveillance of individuals or forced eviction; this is stated in the report and the user manual.

- **Honest evaluation:** the test set was never used for training, model selection or threshold choice; results are reported with the literature’s dataset-level convention, and per-image averages are labelled separately.
- **Reporting negative results:** the defects of our first prototype and the finding that the Canny channel gives no measurable gain are reported openly rather than hidden, and the comparison with published methods is marked as indicative.
- **Transparency:** all code, the trained model and the evaluation outputs are public on GitHub. AI assistance used during code review, debugging and document preparation is disclosed here; all results were produced and verified by the team.

6 Reflection on Individual and Team work (PO(i))

6.1 Individual Contribution of Each Member

Table 11: Individual contributions.

Member	Student ID	Main contributions
Mustasin Rahman	2206104	Literature review; Dataset preparation and splits; PyQt5 GUI.
Fahim Shahriyar	2206114	Network architecture; Training pipeline results, error-type and spatial analyses; Failure-mode study; Bangladesh inference
Md. Ahasan Habib Kawsar	2206119	Network architecture; Training pipeline; results, Previous model debugging and re-training; evaluation and Ablation experiments, Bangladesh inference, video and GitHub repository
Md. Kawsar Ahmed	2206122	Literature review; DSP pre-processing (Grayscale, Gaussian, Canny); Augmentation; DSP visualisations and Post-processing; and GitHub repository

6.2 Mode of TeamWork

The team met weekly to review progress and assign tasks, kept a shared task list, and worked in a single shared code folder that was later published as a Git repository. Every module was reviewed and tested by a second member before it was merged. Long training runs were shared across members’ laptops, and design decisions (for example the switch to native-resolution training and the ablation study) were discussed and recorded in the log book below.

6.3 Diversity Statement of Team

The team combines members with different strengths: signal processing, machine learning, software and user-interface development, and technical writing. Tasks were assigned according to interest and then rotated for reviews, so that every member understood the whole

pipeline. All members had an equal voice in decisions, and disagreements were resolved with experiments rather than opinions.

6.4 Log Book of Project Implementation

Table 12: Project log book.

Date	Milestone achieved	Individual role	Team role	Comments
Week 5	Topic, Proposal and Literature Survey	Fahim: Survey; Ahasan: Scope	All: Topic Selection	Levir-CD & WHU-CD Selected
Week 6	Data Pipeline and Dsp Front-End	Ahasan: Loader, Preprocessing Kawsar: Dsp Feature Engineering (Grayscale, Gaussian Filter Canny)	Review By All	4-Channel Input Created
Week 7	First Siamese U-Net Trained; Loss Ablation; Analysis Scripts	Ahasan, Fahim: Model Customization For 3 Channels To 4 Channels Kawsar: Analyses	All: Result Review	Bangladesh Imagery Collected
Week 8	Training Variants (Simple, Weighted, Fast)	Ahasan, Fahim	-----	V1 At 256×256 Resolution
Week 9	V1 Model (F1 0.771); Full Test Inference; Gui With TTA	Mustasin: GUI; Ahasan: Evaluation	All: Testing	Dhaka Domain-Shift Test
Week 10	Dsp Visualisations for Presentation	Ahasan, Kawsar	Ahsan, Kawsar, Fahim-Slide	Stage-By-Stage Figures
Week 11	Code Review: 3 Defects Found; V2 Re-Trained (≈ 5 H)	Ahasan: Fixes, Training	All: Sanity Check	Test F1 0.912
Week 12	TTA And RGB-Only Ablations; Presentation; GitHub	Mustasin: Ablations; Ahasan & Kawsar: GitHub	Fahim, Kawsar, Ahasan: Slides	All results Comparison

7 Communication to External Stakeholders (PO(j))

7.1 Executive Summary

BUET students build an AI tool that spots new buildings from space. A team of four Electrical and Electronic Engineering students at BUET has developed software that compares two satellite photos of the same area, taken years apart, and automatically highlights

every new building. The tool combines classic image-processing filters with a modern neural network and **the reference system achieves 91.17% F1 for pixel-level building-change detection on the unseen LEVIR-CD test set and 82.85% F1 on the cross-dataset WHU-CD evaluation**. The application is designed for local laptop-class inference; selected training runs used available GPU acceleration. City planners could use it to find unauthorised construction, track the loss of wetlands and map damage after floods, while a human expert always confirms the results.

7.2 User Manual

Requirements: Python 3.10 or newer; macOS (Apple silicon), Windows or Linux; about 2 GB of free disk space (plus 5 GB for the dataset if training).

Installation

1. Download the code: `git clone https://github.com/ahasanhabibkawsar/EEE-312-Group-04-Urban-change-detection-DSP-deep-learning-.git`
2. Create a virtual environment: `python3 -m venv .venv`, then activate it (`source .venv/bin/activate` on macOS/Linux).
3. Install the libraries: `pip install -r requirements.txt`
4. Download `best_model.pth` from the repository's Releases page into `checkpoints/` (and, for the model comparison, `best_model_v1.pth` into `_v1_baseline/checkpoints/` and the RGB-only `best_model.pth` into `checkpoints/ablation_rgb_only/`).
5. Optional check: `python sanity_check.py` (all checks should pass in 1–3 minutes).

Using the desktop application

1. Start the application: `python app.py`. The status bar shows “Model ready” when the model has loaded.
2. Click **Load Before (T1)** and **Load After (T2)** and select two co-registered images of the same area (PNG, JPG or TIFF).
3. Optionally click **Load Ground Truth** to see accuracy figures for a labelled image.
4. Choose options: Test-time augmentation (more accurate, about four times slower), post-processing (clean polygons), Centre crop, and Resolution (select the option matching the image's ground resolution; LEVIR-CD is 0.5 m/pixel).
5. Choose the **Model** (default: v2 – RGB + Canny, final) and where to draw the overlay (**Before** or **After** image).
6. Click **Run Inference**. The probability map, change mask and red overlay appear after a few seconds. **Compare all models** opens a window with every model side by side; **Save comparison** exports it.
7. Move the **Threshold** slider to trade false alarms against missed buildings; **Validation Threshold** restores 0.41 and **Auto Threshold (Otsu)** chooses a value for the current image.
8. Read the changed-area percentage (and F1, IoU, precision and recall if a ground truth was loaded) in the metrics panel.
9. Click **Save View** to export a six-panel PNG figure.

Command-line use: `python train.py` (training; `--resume` to continue, `--no-canny` for the RGB-only model); `python evaluate.py --tta` (test evaluation and reports); `python`

test_all_inference.py --data-dir <folder> (predict every pair in a folder with A/ and B/ sub-folders); python compare_models.py (cross-dataset comparison of all models); python save_all_samples.py --data-dir <folder> (one comparison figure per image pair). **Troubleshooting:** if the model file is missing, download it from Releases; if images have different sizes, the after image is resized automatically; for imagery of a different resolution, use the Resolution option.

7.3 Github Link

<https://github.com/ahasanhabibkawsar/EEE-312-Group-04-Urban-change-detection-DSP-deep-learning->

7.4 YouTube Link

<https://youtu.be/-EsIu7B3CKo>

Title: *Urban Change Detection with DSP & Deep Learning | B2.04 | EEE 312 (Jan 2026) Project | Dept of EEE, BUET*

7.5 Google Drive Link (Alternative Video Link)

<https://drive.google.com/drive/folders/1PidWs84WgeYZyie0v5PsjVR7Y3fWRIB6>

8 Future Work (PO(I))

- **Local dataset:** collect and annotate bi-temporal high-resolution images of Dhaka, Purbachal and other Bangladeshi cities to build a region-specific benchmark, so that the old and new models can be scored on local data.
- **Domain adaptation:** fine-tune the LEVIR-CD model on local data and study unsupervised domain adaptation for dense, high-rise scenes.
- **Adaptive DSP features:** replace fixed Canny thresholds with learnable, multi-scale edge filters initialised from Sobel/Gabor kernels, and repeat the ablation with several random seeds.
- **Semantic change detection:** classify the type of change (new construction, demolition, extension) and add roads and vegetation.
- **Multi-temporal analysis:** use time series of more than two images to suppress seasonal pseudo-changes.
- **Deployment:** faster inference for city-scale scenes and a GIS plug-in (e.g. QGIS) with GeoTIFF input and polygon export.

9 References

- [1] A. Singh, "Digital change detection techniques using remotely-sensed data," *Int. J. Remote Sensing*, vol. 10, no. 6, pp. 989–1003, 1989.
- [2] R. C. Daudt, B. Le Saux, and A. Boulch, "Fully convolutional Siamese networks for change detection," in *Proc. IEEE Int. Conf. Image Processing (ICIP)*, 2018, pp. 4063–4067.
- [3] H. Chen and Z. Shi, "A spatial-temporal attention-based method and a new dataset for remote sensing image change detection," *Remote Sensing*, vol. 12, no. 10, p. 1662, 2020.
- [4] H. Chen, Z. Qi, and Z. Shi, "Remote sensing image change detection with transformers," *IEEE Trans. Geoscience and Remote Sensing*, vol. 60, pp. 1–14, 2022.
- [5] W. G. C. Bandara and V. M. Patel, "A transformer-based Siamese network for change detection," in *Proc. IEEE Int. Geoscience and Remote Sensing Symp. (IGARSS)*, 2022, pp. 207–210.
- [6] S. Ji, S. Wei, and M. Lu, "Fully convolutional networks for multisource building extraction from an open aerial and satellite imagery data set," *IEEE Trans. Geoscience and Remote Sensing*, vol. 57, no. 1, pp. 574–586, 2019.
- [7] D. Marr and E. Hildreth, "Theory of edge detection," *Proc. Royal Society of London B*, vol. 207, no. 1167, pp. 187–217, 1980.
- [8] J. Canny, "A computational approach to edge detection," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. PAMI-8, no. 6, pp. 679–698, 1986.
- [9] S. Xie and Z. Tu, "Holistically-nested edge detection," in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2015, pp. 1395–1403.
- [10] LEVIR-CD: A large-scale remote sensing building change detection dataset. [Online]. Available: <https://justchenhao.github.io/LEVIR/>
- [11] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. New York, NY, USA: Pearson, 2018.
- [12] ITU-R Recommendation BT.601-7, "Studio encoding parameters of digital television for standard 4:3 and wide-screen 16:9 aspect ratios," International Telecommunication Union, 2011.
- [13] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [14] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Proc. MICCAI*, 2015, pp. 234–241.
- [15] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [16] F. Milletari, N. Navab, and S.-A. Ahmadi, "V-Net: Fully convolutional neural networks for volumetric medical image segmentation," in *Proc. Int. Conf. 3D Vision (3DV)*, 2016, pp. 565–571.
- [17] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2019.
- [18] D. H. Douglas and T. K. Peucker, "Algorithms for the reduction of the number of points required to represent a digitized line or its caricature," *The Canadian Cartographer*, vol. 10, no. 2, pp. 112–122, 1973.
- [19] N. Otsu, "A threshold selection method from gray-level histograms," *IEEE Trans. Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, 1979.
- [20] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [21] G. Bradski, "The OpenCV library," *Dr. Dobb's Journal of Software Tools*, 2000.